# KIẾN TRÚC POC ĐÁNH GIÁ CÂU TRẢ LỜI

**Mục tiêu.** Đối chiếu câu trả lời gốc với tiêu chí và thang điểm của từng năng lực; chỉ ra mức đạt, phần chưa đạt và căn cứ. Một tiêu chí thiếu căn cứ được giữ riêng, không tự đổi thành điểm 0. Đây là PoC cho một câu hỏi đã chọn, không kết luận năng lực tổng thể.

**Phạm vi mã.** Một file khởi chạy **source/main.py**. Các file còn lại thuộc Model, Controller, Service hoặc View. Báo cáo PDF ở **source/reports/** là tài liệu, không có logic chạy. File **source/__init__.py** là dấu mốc package rỗng của Python.

## 1. Luồng xử lý từ lúc nhận câu trả lời

| Bước | Lệnh gọi và đầu ra |
|---|---|
| 1. Khởi động | **main.main()** gọi **createController()** để ghép repository, interpreter factory và service; **readArguments()** nhận đường dẫn, provider và định dạng hiển thị. |
| 2. Nhận yêu cầu | **runView()** đưa request vào **EvaluationController.submitAnswer()**. **EvaluationRequest.checkRecording()** kiểm tra cấu trúc và cặp provider/recording. |
| 3. Đọc bài gốc | Controller gọi **EvaluationService.evaluateFiles()**, rồi **loadInputs()**. Repository dùng **loadInput()**, **loadAnswer()** (nếu có bài mới), **loadRecording()** (nếu chạy mẫu). **readTextFile()** đọc UTF-8 có giới hạn. Bài mới tạo ID/revision mới, không sửa file đầu vào. |
| 4. Kiểm tra chuẩn | **assessAnswer()** gọi **createEvaluation()**; **validateInputs()** kiểm tra liên kết question/answer/rubric, ID duy nhất; **validateLevels()** kiểm tra thang điểm tăng dần, mức nền và khả năng đạt từng mức. |
| 5. Trích xuất và làm rõ | **evaluateCriteria()** gọi **interpreter.interpret()**. Gemini dùng **systemPrompt()**, **promptPayload()**, **buildRequest()**, **parseResponse()** để tạo các criterionCheck gồm met/unmet/insufficientEvidence và bằng chứng. Recorded đọc diễn giải đã lưu sau khi so **inputDigest()**. Mô hình chỉ diễn giải, không cho điểm. |
| 6. Kiểm tra bằng chứng | **validateInterpretation()** yêu cầu đúng rubric và criterion; **validateCheck()**, **validateQuote()**, **validateOmission()** xác minh trích dẫn nguyên văn, offset, phần thiếu và mơ hồ. Nội dung chưa rõ không bị suy diễn thành câu trả lời sai. |
| 7. Chấm và phản hồi | **scoreChecks()** dùng **findLevel()** chọn mức thấp/cao khả dĩ từ tiêu chí đã xác nhận và chưa rõ. Nếu hai mức khác nhau, score=null. **generateFeedback()** gọi **buildFeedback()**; **validateResults()** đối chiếu lại điểm, lý do, liên kết và phản hồi. |
| 8. Lưu và trình bày | **readReport()** tạo báo cáo có snapshot đầu vào, hash và phiên bản provider/prompt. **JsonEvaluationRepository.saveReport()** ghi qua file tạm và chỉ tạo file đích mới. Controller chuyển lỗi thành **EvaluationFailure**; **renderReport()** hoặc JSON được View in ra. |

## 2. Ranh giới giữa các lớp

| Tầng | Trách nhiệm và lý do |
|---|---|
| Model | Cấu trúc dữ liệu, ràng buộc trường và nơi lưu. Không quyết định điểm; giữ bản gốc, liên kết và dữ liệu có thể kiểm tra. |
| Service | Quy tắc đánh giá, kiểm tra bằng chứng, diễn giải AI, tính điểm và tạo phản hồi. Có thể chạy độc lập với giao diện. |
| Controller | Kiểm tra hình dạng request, gọi Service, chuyển ngoại lệ thành lỗi có mã. Không đọc file hay chấm điểm. |
| View | Nhận tham số, gọi Controller, trình bày kết quả/lỗi. Không đọc file, gọi AI hoặc tính điểm. |
| main.py | Ghép các phụ thuộc cụ thể vào interface; là điểm vào duy nhất của PoC. |

## 3. File và hàm: khởi chạy, Controller, View

| File | Hàm / lớp và nhiệm vụ |
|---|---|
| source/main.py | **createInterpreter**: chọn Recorded hoặc Gemini, không chấm điểm. **createController**: tạo repository, service, controller. **main**: đọc tham số và chạy View; trả exit code. |
| source/Control/evaluationSchemas.py | **EvaluationRequest**: đường dẫn và provider; **checkRecording**: ràng buộc recording. **EvaluationFailure**: mã lỗi, thông báo, khả năng thử lại. **EvaluationResponse**: báo cáo hoặc lỗi; **checkOutcome**: không cho trạng thái nửa vời. |
| source/Control/evaluationController.py | **EvaluationController.submitAnswer**: xác thực request, gọi evaluateFiles, trả EvaluationResponse; dịch lỗi nghiệp vụ, Pydantic và lỗi file thành lỗi có cấu trúc. **__init__**: nhận Service để dễ thay thế. |
| source/view/evaluationView.py | **readArguments**: tham số terminal. **runView**: tạo request, gọi Controller, in JSON/text và lỗi. **renderReport**: định dạng điểm, mức khai báo, nhận xét và trích dẫn; không tính toán lại. |

## 4. File và hàm: Model

| File | Hàm / lớp và nhiệm vụ |
|---|---|
| Model/answerData.py | **FrozenData**: dữ liệu bất biến, cấm trường lạ. **Question**: câu hỏi, yêu cầu, ngữ cảnh. **Criterion**: điều kiện chấp nhận, quy tắc xem xét, quyền kết luận từ phần thiếu. **ScoreLevel**: điểm, mô tả và điều kiện đạt. **Rubric**: nhóm tiêu chí cho một năng lực. **Answer**: nguyên văn, ID/revision. **EvaluationInput**: ghép câu hỏi, rubrics, bài làm và cờ minh họa. |
| Model/evaluationData.py | **Evidence**: quote/omission/ambiguity và offset. **CriterionCheck**, **RubricReview**, **Interpretation**: diễn giải từng tiêu chí. **EvaluationResult** và **checkScore**: điểm hoặc null theo trạng thái. **Feedback**: phản hồi. **AnswerEvaluation**: trạng thái trung gian. **EvaluationReport**: báo cáo hoàn chỉnh/snapshot. **RecordedReview**: diễn giải mẫu kèm hash. |
| Model/repositories/evaluationRepository.py | **EvaluationRepository**: Protocol gồm loadInput, loadAnswer, loadRecording, resolveOutput, saveReport. Service dùng interface này để không phụ thuộc JSON hay đường dẫn cụ thể. |
| Model/repositories/jsonEvaluationRepository.py | **readTextFile**: đọc UTF-8, giới hạn 1 MB. **writeNewReport**: ghi file tạm rồi tạo file đích không ghi đè. **JsonEvaluationRepository**: cài năm thao tác đọc/chọn đường dẫn/lưu của Protocol; **resolveOutput** sinh tên mới khi không chỉ định. |

## 5. File và hàm: Service cốt lõi

| File | Hàm / lớp và nhiệm vụ |
|---|---|
| service/evaluationService.py | **EvaluationService.loadInputs**: đọc input và tạo bản trả lời mới nếu có file bổ sung. **evaluateFiles**: điều phối đọc, chọn interpreter, đánh giá và lưu. **assessAnswer**: thứ tự đánh giá thuần trong bộ nhớ. **createEvaluation**: tạo trạng thái/kiểm tra đầu vào. **evaluateCriteria**: lấy diễn giải, xác thực, tính từng rubric. **generateFeedback**: sinh phản hồi từ kết quả. **validateResults**: kiểm tra lại toàn bộ liên kết, mức điểm và phản hồi. **readReport**: đóng gói báo cáo có hash và metadata. |
| service/rubricRules.py | **requireUnique**: phát hiện ID lặp. **validateInputs**: kiểm tra liên kết câu hỏi, đáp án, rubric. **validateLevels**: kiểm tra thứ tự, mức nền, ngưỡng và mức có thể đạt. **findLevel**: chọn mức cao nhất thỏa điều kiện. **scoreChecks**: so mức chắc chắn với mức tốt nhất có thể; trả điểm hoặc null và lý do. |
| service/evidenceReview.py | **invalidEvidence**: lỗi thống nhất. **validateQuote**: trích dẫn và vị trí chính xác trong nguyên văn. **validateOmission**: phần thiếu và quyền kết luận theo rubric. **validateCheck**: loại bằng chứng phù hợp trạng thái. **validateInterpretation**: đúng một review/rubric và check/criterion, theo thứ tự rubric. |
| service/feedbackBuilder.py | **buildFeedback**: biến kết quả và bằng chứng đã xác thực thành giải thích, mức đáp ứng và danh sách điểm chưa đạt; không gọi AI lần hai. |
| service/evaluationErrors.py | **EvaluationError.__init__**: giữ mã lỗi, thông báo tiếng Anh và cờ retryable để Controller không nhầm lỗi hạ tầng với năng lực người học. |

## 6. File và hàm: nguồn diễn giải

| File | Hàm / lớp và nhiệm vụ |
|---|---|
| service/providers/evidenceInterpreter.py | **EvidenceInterpreter.interpret**: Protocol cho nguồn diễn giải. **inputDigest**: SHA-256 của snapshot đầu vào, dùng khóa bản diễn giải mẫu với đúng câu hỏi/rubric/bài làm. |
| service/providers/geminiInterpreter.py | **GeminiSettings**: cấu hình giữ kín API key khi repr. **readGeminiSettings**: lấy key/model/timeout từ môi trường. **GeminiInterpreter.__init__**: kiểm tra cấu hình. **buildRequest**: tạo request JSON có schema và giới hạn dung lượng. **interpret**: gọi dịch vụ, phân loại quota/HTTP/timeout/mạng. **parseResponse**: chỉ nhận một kết quả JSON hoàn chỉnh, đúng schema. |
| service/providers/recordedInterpreter.py | **RecordedInterpreter.__init__**: giữ bản diễn giải có sẵn. **interpret**: so inputHash rồi trả interpretation; không giả vờ đây là lần chấm AI trực tiếp. |
| service/prompts/evaluationPrompt.py | **systemPrompt**: yêu cầu diễn giải theo criterion, dẫn nguyên văn và không tự cho điểm. **promptPayload**: chỉ gửi question, rubrics cần xem và answer, loại phần thang điểm khỏi đầu vào AI. **promptVersion**: gắn phiên bản cho báo cáo. |

## 7. Các file có sẵn và file kiểm tra

**Model/ids.py, Model/question.py, Model/evaluation.py** là hợp đồng P03 có trước. Chúng giữ ID và các dataclass cho ngân hàng câu hỏi/kết quả của nhóm; PoC chỉ tái sử dụng kiểu ID, không tự đổi hợp đồng một rubric/câu và điểm theo criterion. Sự khác biệt với schema **answerEvaluation/1** được ghi ở docs/data_contract.md. Các file **__init__.py** là dấu mốc package, không chứa nghiệp vụ.

**test/answerEvaluationTest.py**: kiểm tra mức điểm, căn cứ, trạng thái và lỗi cuối luồng. **test/evaluationCommandTest.py**: lệnh chạy, định dạng hiển thị, không ghi đè và hash bài mới. **test/geminiInterpreterTest.py**: request/response và lỗi giả lập, không gọi mạng thật. **test/evaluationArchitectureTest.py**: ranh giới import, thay repository, ghép lớp và vị trí mã. **test/test_environment.py** là smoke test web có sẵn của nhóm, không thuộc PoC này.

## 8. Quyết định thiết kế và giới hạn

**Gộp trùng.** Đã bỏ entry phụ source/evaluateAnswer.py, file cấu hình cấp source, file trạng thái Service riêng và file CLI View riêng. Chỉ còn một quy trình chấm trong evaluationService.py; rubric, chứng cứ và phản hồi vẫn tách thành hàm nhỏ vì mỗi file có một quy tắc khác nhau.

**OOP.** Dùng lớp ở nơi có trạng thái hoặc phụ thuộc cần thay: Controller, Service, Repository và Interpreter. Các phép kiểm tra và tính mức điểm nên là hàm thuần để dễ đọc, chạy độc lập và debug. Không cần chuyển toàn bộ thuật toán sang class.

**Giới hạn.** Mẫu là nội dung tự soạn chưa được duyệt; recorded không phải đánh giá AI thật. Gemini chỉ đưa ra diễn giải có cấu trúc, và bằng chứng được đối chiếu nguyên văn, nhưng chất lượng nhận định ngữ nghĩa vẫn cần người rà soát trước khi dùng với người học. PoC không thực thi code trong bài làm, không suy ra năng lực tổng thể, và không tự tích hợp vào hợp đồng P03 khi chưa thống nhất schema.